Registered evidence search — Abacus release handoff

Scope and dependency

This change builds on `evidence-pipeline-reviewed-search` (PR #5). It adds actual PostgreSQL vector retrieval to the Next.js application; it does not depend on copying the private SQLite index from PR #2. Use the existing BioVeracity Abacus app. The registered search route now includes checked passages for normal accounts, alongside existing place results. Institutional reports and other institutional capabilities keep their existing access rules.

The user-facing routes are `/search` (top five evidence results alongside places), `/evidence` (up to 30 results, council/date filters, meaning+keywords or keyword-only, and event-date arrangement), and authenticated `/api/evidence/search`. Anonymous users see a sign-in invitation; no evidence or provider call is made for them. Server guards apply to every search entry point. No client-supplied role, embedding, model or verification status is trusted.

Retrieval and integrity

- The embedding unit is the active reviewed claim plus its exact supporting excerpt, not raw unreviewed documents. The review's publication confirmation is required upstream. Reviewer identity, private review notes and raw pending source sections are not sent to the provider or returned in results.
- The default embedding model is `text-embedding-3-small`, with 1536 dimensions; `text-embedding-3-large` at 1536 is also supported. Provider, model, dimension and text recipe form an explicit embedding space. Switching models requires indexing that new space. Existing vectors cannot be silently reused across models or corrections.
- `yarn index:evidence` indexes a bounded batch of 16 current, reviewed, missing claims. A database advisory lock prevents overlapping worker transactions. Failures roll back and return a nonzero exit; reruns retry missing embeddings without creating duplicates. Embeddings belong to immutable review IDs. Keep this scheduled job running to pick up subsequent reviews automatically. In this managed deployment the worker (`scripts/index-evidence.ts`) is adapted to exit 0 as a clean no-op when vector search is not configured, so a recurring managed schedule never errors while the feature is dormant.
- Every query selects current source versions before checking status, council, dates or relevance. Pending replacements, withdrawals and new reviews cannot reuse older embeddings in results. Search uses pgvector cosine distance plus PostgreSQL full-text search, fused by reciprocal rank (k=60). This is relevance, never an evidence-quality or verification score.
- Exact vector search currently applies eligibility filters before ranking. This avoids approximate-index filtering omissions, but is not a tested national-scale latency guarantee. Each channel contributes up to 100 candidates and returns 30 fused results. Measure filtered query latency/recall on the intended corpus before adding HNSW or claiming national-scale performance.
- Date filters use known event days only. Event-date arrangement sorts the retrieved top 30, with unknown/coarse dates afterwards; it is explicitly not an exhaustive timeline. No relation or causation between separate records is generated.

- The interface reports meaning+keyword versus keyword-only mode and indexed/eligible claim counts within the filters. Counts describe the retained reviewed collection, not complete council coverage. Provider/config/database-vector failures fall back visibly to keyword search. An atomic database quota limits embedding queries to 20 per account per minute across app processes; keyword-only searches remain available. This is a per-account control, not a platform-wide spend ceiling; configure the provider project budget and monitor usage before launch.

Deploy through the existing Abacus app

1. Reconcile this branch, PR #5 and the currently deployed commit. Establish the actual Abacus release command, checkpoint and rollback mechanism. There is no confirmed callable Abacus deploy connection in this workspace.
2. This managed deployment uses Yarn exclusively. `node_modules` and `yarn.lock` are platform symlinks into the shared runtime; the two test-only devDependencies were added with `yarn add -D @electric-sql/pglite@0.5.8 @electric-sql/pglite-pgvector@0.0.9` (0.0.9 matches PGlite 0.5.8), which updates the platform-managed lock in place. Then run `yarn prisma generate`, `yarn typecheck` and `yarn test:evidence`.
3. Use isolated staging PostgreSQL with pgvector available. Resolve the existing migration baseline as described in the pipeline handoff before applying any migrations. Apply `0001_evidence_pipeline` and the new additive `0002_evidence_vectors` migration in staging. The latter creates `EvidenceEmbedding` and `EvidenceSearchQuota` and enables the pgvector extension. Do not use `db push`, reset, seed, or recreate the live database. If pgvector is unavailable, stop vector activation and establish supported database hosting; do not report keyword fallback as vector search.
4. In staging server/worker secret settings, set `BIOVERACITY_EVIDENCE_ENABLED=true`, `BIOVERACITY_VECTOR_ENABLED=true`, `BIOVERACITY_EMBEDDING_MODEL=text-embedding-3-small`, and `OPENAI_API_KEY` for the approved embedding project. No key belongs in browser variables, GitHub, chat or logs. Meaning-based queries and reviewed claim/excerpt text are sent to OpenAI; document this in the product’s data-processing notice before activation. No API key was accessed or provider charge incurred by these tests.
5. Run the source-to-review staging check from the pipeline handoff with a real permitted public source. Run `yarn index:evidence` in `nextjs_space` and inspect its indexed count. Schedule that same bounded command using a managed scheduled task; the managed scheduler’s minimum interval is one hour, so schedule an hourly indexing sweep (not once per minute as originally drafted). Record two successful scheduled executions, a restart and a subsequent newly reviewed record becoming indexed. Do not substitute a ChatGPT research automation for this worker.
6. Test a newly registered account through real login. Submit a natural-language question at the main search bar, read its cited passage, follow the evidence filter link, filter by council and event day, and select keyword-only mode. Verify guests cannot read the evidence API, ordinary accounts cannot access admin review or institutional reports, and source review notes never appear in the browser response. Check desktop and mobile layouts.
7. Evaluate at least 20 human-labelled real-source queries across the available Cambridge/Peterborough collection and other covered areas: paraphrases without shared words, exact names, precise dates, unrelated questions, missing evidence and differing source statements. Record recall@10, reciprocal rank, keyword baseline, latency and actual indexed coverage. Review semantic-only hits for usefulness; synthetic embeddings prove wiring, not semantic quality. Do not claim all 417 councils, rivers or ports are searchable without an audited coverage ledger.
8. Test a new unreviewed source version, a withdrawn claim and a corrected review after indexing. Old results must disappear immediately, and corrected claims must remain keyword-searchable

while awaiting their own embedding. Simulate provider outage and quota exhaustion; the UI must say keyword-only. Confirm background retries and monitor failure/coverage counts.

9. After staging gates, take the actual production checkpoint/backup and release through the confirmed Abacus route. Use production-only credentials and enable the index schedule. Repeat the real account and source-to-search checks on bioveracity.com. Only these checks establish a live capability.

Rollback

Set `BIOVERACITY_VECTOR_ENABLED=false` and stop the index job to retain reviewed keyword search. Set `BIOVERACITY_EVIDENCE_ENABLED=false` to hide the whole evidence feature. Restore the confirmed previous app checkpoint if needed. Retain the additive evidence/audit/vector tables; do not drop evidence history as rollback. Keep the prior integrity fixes or pause legacy publication while reverting.

Local evidence of correctness

Prisma generation and the explicit TypeScript check passed. All 12 TypeScript tests passed, including under `TZ=Europe/Dublin`. The suite executes the application’s actual SQL and both additive migrations in isolated PGlite PostgreSQL with the pgvector extension. Synthetic vectors test non-keyword matches, fusion, missing-index visibility, model isolation, current-version suppression, withdrawal, corrected-review reindexing, idempotent indexing, failed-worker rollback, dates, councils, registered/anonymous access, privacy boundaries, provider failures and quota fallback. Provider-contract tests validate dimensions/model/indices, finite nonzero vectors, input bounds and response ordering. Existing TypeScript pipeline tests remain included.

No live provider retrieval quality, Abacus sign-in session, production migration/deploy, persistent scheduler or national coverage has been verified by these local tests. Honeycomb graph traversal, exhaustive chronology pagination and ranking evaluation remain separate from this completed application wiring.

Implementation references: OpenAI embeddings API, pgvector, PGlite extensions.